

PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
Facultad de Ingeniería
IIC2613 Inteligencia Artificial
2026-2

Tarea 1
Aprendizaje Automático y Reforzado

Fecha de entrega: Lunes 5 de Octubre a las 23:59 hrs

Aspectos generales

Formato y plazo de entrega

El formato de entrega corresponde a un zip que contenga las carpetas `01 - DCCubos` y `02 - DCCUber` con sus archivos correspondientes según lo explicado en cada parte de la tarea. El lugar de entrega es por canvas. Por último, recuerda que los cupones de atraso corresponden a días **reales** (hábiles o no) extra.

Integridad académica

Este curso se adhiere al Código de Honor establecido por la universidad, el cual tienes el deber de conocer como estudiante. Todo el trabajo realizado en esta tarea debe ser **totalmente individual**. Las dudas se deben hacer al cuerpo docente a través de las *issues* en GitHub.

Todo material obtenido de internet debe ser citado correctamente (APA, ICONTEC o IEEE). Cualquier falta a la ética o a la integridad académica será sancionada con la reprobación del curso y los antecedentes serán entregados a la Dirección de Pregrado.

Uso de agentes inteligentes

El uso de agentes inteligentes —como ChatGPT, Gemini, Copilot, Claude u otros— está permitido, siempre y cuando indiques al inicio de la respuesta si los utilizaste y para qué, adjuntes en un anexo los *prompts* y respuestas relevantes, y modifiques sustancialmente sus respuestas para conectarlas con **tus propios resultados**.

Esta tarea está diseñada para que el análisis dependa de tus ejecuciones concretas: gráficos, métricas y matrices de confusión. Las respuestas genéricas que no hagan referencia a esos resultados no serán consideradas.

Comentarios adicionales

Es fundamental que pongas énfasis en las justificaciones, cuidando la redacción y la ortografía, y manteniendo el código ordenado y comentado. Las respuestas que solo presenten resultados o código, sin explicar el procedimiento ni interpretar los resultados, no serán consideradas.

Recursos para realizar la tarea

Pandas

Pandas permite trabajar con datos tabulares mediante objetos `DataFrame`. Para comenzar tu análisis, te pueden resultar útiles los métodos `head`, `info`, `describe`, `isna`, `duplicated` y `value_counts`. También puedes utilizar `groupby` para estudiar una variable dentro de grupos comparables y `merge` para unir tablas mediante una llave común.

En esta tarea, la columna `filename` es la llave que relaciona las imágenes con su ficha técnica. Por lo tanto, debes cruzar ambas fuentes mediante esa columna y **no por la posición de las filas**.

Scikit-learn y scikit-image

Scikit-learn provee los modelos clásicos que utilizarás, además de herramientas para dividir los datos, escalar variables, construir *pipelines* y evaluar clasificadores. En particular, revisa la documentación de:

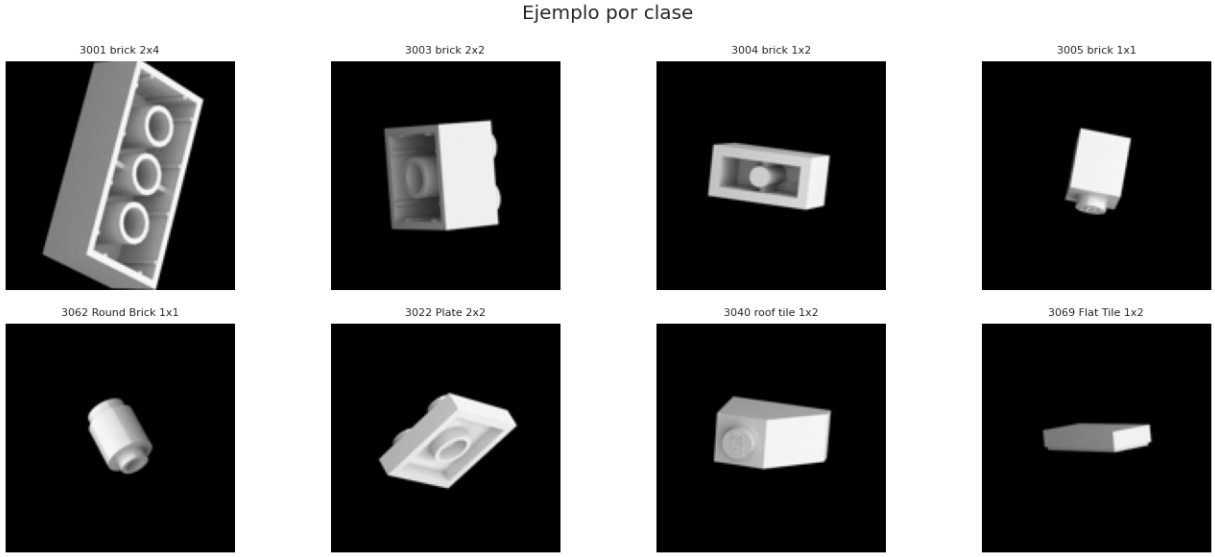
- `train_test_split`, `StandardScaler` y `Pipeline`;
- `SVC`, `RandomForestClassifier`, `DecisionTreeClassifier` y las métricas de clasificación.

Para describir la forma de las piezas puedes apoyarte en scikit-image. Su documentación contiene implementaciones y referencias para descriptores como HOG y LBP, además de propiedades geométricas de regiones. Recuerda que toda característica investigada debe estar acompañada por una fuente apropiada.

Google Colab y descarga de datos

Se recomienda desarrollar la tarea en Google Colab. El notebook base descarga el dataset con `kagglehub`; son aproximadamente **1 GB**, por lo que la descarga puede tardar cerca de un minuto y se realiza una vez por sesión. Si Kaggle solicita autenticación, ejecuta `kagglehub.login()` en una celda aparte antes de descargar los datos. Necesitarás una cuenta de Kaggle y un *API token*.

1. DCCubos (6.0 puntos)



1.1. Introducción

Es tu primer día de práctica en **DCCubos**, una fábrica de bloques de construcción que busca automatizar la clasificación de las piezas que circulan por su línea de producción. Hasta ahora, una persona con años de experiencia reconoce cada pieza de un vistazo y la separa antes de que sea empaquetada. Sin embargo, la cantidad de piezas ha aumentado y la fábrica necesita una solución que pueda operar de manera consistente.

Dentro de la empresa existen dos propuestas. El equipo de TI instaló una cámara sobre la cinta y sostiene que la forma visible en cada imagen contiene toda la información necesaria. Por otra parte, el equipo de operaciones prefiere utilizar las fichas técnicas del inventario: peso, dimensiones, precio y otros datos registrados para cada pieza. El problema es que el sistema de inventario es antiguo y contiene valores faltantes, texto inconsistente y errores de registro.

Como estudiante de Inteligencia Artificial, tu misión será construir y comparar ambas alternativas. No bastará con obtener una métrica alta: deberás explicar qué información aprovecha cada enfoque, en qué casos falla y si una combinación de ambos mundos justifica su costo adicional.

1.2. Objetivo general

El objetivo de esta tarea es desarrollar modelos de clasificación supervisada capaces de reconocer ocho tipos de piezas a partir de imágenes monocromas y datos tabulares. Trabajarás con modelos clásicos **sin *deep learning*** y compararás tres escenarios:

- **Track A — Imagen:** utiliza características construidas a partir de las imágenes.
- **Track B — Tabular:** utiliza únicamente la ficha técnica del inventario.
- **Track C — Combinado:** reúne las mejores características de los dos escenarios anteriores.

Al finalizar, deberás responder la pregunta central de la fábrica: **¿conviene clasificar a partir de la imagen, de los datos tabulares o de una combinación de ambos?**

1.3. Dataset

Para la parte visual se utilizará el dataset *Images of LEGO Bricks* de Kaggle. Este contiene renders monocromos de 200×200 píxeles en formato PNG. Trabajarás con el subconjunto de ocho piezas mostrado

en la Tabla 1. El notebook descarga y organiza automáticamente las imágenes en la carpeta `data/`.

Pieza	Descripción
`3001 brick 2x4`	Ladrillo clásico, 8 *studs*
`3003 brick 2x2`	Ladrillo cuadrado, 4 *studs*
`3004 brick 1x2`	Ladrillo alargado, 2 *studs*
`3005 brick 1x1`	Ladrillo unitario, 1 *stud*
`3022 Plate 2x2`	Placa baja 2x2, 4 *studs*
`3062 Round Brick 1x1`	Ladrillo redondo, 1 *stud*
`3040 roof tile 1x2`	Pieza inclinada (*slope*)
`3069 Flat Tile 1x2`	Loseta lisa, sin *studs*

Cuadro 1: Piezas a clasificar. Los nombres reproducen exactamente la columna `label` del dataset; el uso de mayúsculas y minúsculas es inconsistente en la fuente original.

Para la parte tabular se entrega el archivo `piezas_dataset.csv`, que contiene una ficha técnica por imagen con variables como peso, altura, número de *studs*, superficie, precio y categoría. La columna `filename` permite relacionar cada fila con su imagen correspondiente.

El notebook `Tarea1_LEGO.ipynb` (que se encuentra dentro del directorio `DCCubos`) incluye la descarga, organización y exploración inicial de las imágenes, además de una segmentación de referencia y la carga de la ficha técnica. **El resto del desarrollo lo debes implementar tú**, agregando las celdas que necesites.

1.4. Actividades

A continuación se describen las actividades que debes realizar. Recuerda que toda decisión debe estar justificada y que todo resultado debe ser analizado. Si se entrega código sin explicación, este no será considerado.

ADVERTENCIA — Partición única: define una sola partición de entrenamiento y prueba, y reutiliza los mismos índices, proporción y semilla en los tres *tracks*. Las comparaciones entre los Tracks A, B y C solo son válidas si los modelos se evalúan sobre exactamente el mismo conjunto de prueba. La selección de hiperparámetros debe realizarse con validación cruzada o con un subconjunto de validación obtenido desde el conjunto de entrenamiento; el conjunto de prueba se reserva para la evaluación final. Fija también el `random_state` de los modelos para que tus resultados sean reproducibles.

1.4.1. Clasificación a partir de imágenes (2.5 puntos)

Las imágenes son monocromas, por lo que la forma es la principal fuente de información discriminante. En esta parte deberás construir características visuales, preparar los datos y comparar modelos clásicos.

Extracción de características (1.0 punto). A partir de las imágenes y de la máscara de segmentación entregada, construye tus propias características. Para ello, debes:

- Investigar en la literatura características que permitan describir imágenes o formas. Se sugiere revisar LBP (*Local Binary Patterns*), Momentos de Hu y HOG (*Histogram of Oriented Gradients*), aunque puedes utilizar otras.
- Respaldar cada característica utilizada con al menos una fuente citada: artículo, libro o documentación oficial.
- Incluir al menos dos características geométricas derivadas de la silueta o máscara de la pieza.
- Generar gráficos, por ejemplo, histogramas por clase, que permitan estudiar el poder discriminativo de las características elegidas.

Normalización y división de datos (0.6 puntos). Divide los datos en entrenamiento y prueba en la proporción que estimes apropiada, procurando que ambos conjuntos sean representativos de las ocho clases. Ajusta el escalado usando solamente los datos de entrenamiento, idealmente dentro de un *pipeline*, y aplica luego la transformación al conjunto de prueba. Recuerda reutilizar esta misma partición en los Tracks B y C.

Entrenamiento y comparación de modelos (0.9 puntos). Entrena los modelos **SVM**, **Random Forest**, **Árbol de Decisión** y un **cuarto modelo a tu elección**. Investiga este cuarto modelo y justifica por qué podría ser útil para clasificar imágenes. Para cada modelo, prueba **tres combinaciones de hiperparámetros**; en total, debes comparar **12 configuraciones** mediante validación cruzada o un conjunto de validación. Define estas combinaciones una sola vez y reutilízalas en los tres *tracks*. Registra el *accuracy* de validación de cada configuración, selecciona la mejor de cada modelo y evalúa esas configuraciones finales sobre el conjunto de prueba. Presenta los resultados mediante una tabla y gráficos.

Preguntas de análisis. Responde en `respuesta.pdf`:

a) ¿Qué mide cada característica y por qué ayuda (o no) a distinguir estas piezas? Relaciona tu respuesta con la forma concreta de las piezas y con los gráficos que generaste.

b) ¿Qué tipo de escalado utilizaste y por qué? Explica por qué es especialmente relevante para modelos basados en distancias o márgenes, como SVM o KNN, y casi indiferente para modelos basados en árboles.

c) Justifica la proporción elegida para la división de datos y explica cómo aseguraste que los conjuntos fueran representativos de todas las clases. Describe además tu estrategia de validación y por qué el conjunto de prueba no debe utilizarse para elegir hiperparámetros.

d) Analiza los resultados apoyándote en gráficos. ¿Qué modelo obtuvo el mejor desempeño y por qué? Relaciona tu respuesta con el funcionamiento de cada algoritmo.

e) ¿Cómo afectó el cambio de hiperparámetros dentro de cada modelo? Explica al menos un caso concreto.

f) Indica cuál fue el mejor y el peor modelo o configuración, y justifica tu respuesta.

NOTA: El no responder las preguntas de análisis implica un descuento, ya que hacen referencia al trabajo realizado en este item.

1.4.2. Clasificación a partir de datos tabulares (2.0 puntos)

En esta parte utilizarás `piezas_dataset.csv`, una ficha técnica asociada a cada imagen. Los datos provienen de un sistema de inventario y pueden contener valores faltantes, formatos inconsistentes, variables irrelevantes y observaciones atípicas.

Análisis del dataset (0.4 puntos). Explora y describe el conjunto de datos. Identifica el tipo y rango de cada variable, los valores faltantes, posibles inconsistencias y observaciones atípicas. Apoya tus hallazgos con gráficos y tablas.

Limpieza y preprocesamiento (0.6 puntos). Limpia el dataset y prepara las variables para los modelos. Según corresponda, deberás decidir cómo corregir o eliminar valores, codificar variables categóricas, imputar datos faltantes y escalar las variables numéricas. Las transformaciones que aprendan parámetros deben ajustarse usando solamente el conjunto de entrenamiento.

Selección de características (0.4 puntos). Realiza una selección de características justificada. Identifica variables redundantes, irrelevantes o que entreguen información que no estaría disponible al momento de realizar la predicción.

ADVERTENCIA — Columna `category`: esta variable describe casi directamente la forma de la pieza que el modelo intenta predecir. En el escenario planteado, esa información no estaría disponible antes de clasificar una pieza. Por lo tanto:

- El resultado principal del Track B, utilizado en la comparación con los otros *tracks*, debe obtenerse **sin** `category`.
- Puedes incluirla como un experimento adicional, siempre que reportes por separado el *accuracy* con y sin `category`, y expliques la diferencia.

Presentar como resultado principal un único valor obtenido con `category` se considerará un error metodológico.

Entrenamiento y comparación de modelos (0.6 puntos). Entrena nuevamente SVM, Random Forest, Árbol de Decisión y el cuarto modelo elegido en el Track A. Explica qué propiedades de este último resultan apropiadas, o limitantes, para datos tabulares. Reutiliza las mismas tres combinaciones de hiperparámetros por modelo, compáralas mediante la misma estrategia de validación y evalúa en prueba solo las configuraciones finales.

Preguntas de análisis. Responde en `respuesta.pdf`:

a) Resume lo observado durante la exploración. ¿Qué problemas o particularidades presenta cada columna?

b) Justifica cada decisión de limpieza y preprocesamiento. ¿Qué habría ocurrido si no la aplicabas?

c) Explica tu criterio de selección y cómo evitaste el *data leakage*. ¿Existían columnas demasiado reveladoras o irrelevantes? Cuantifica cuánto aumenta el *accuracy* al incluir `category` y explica por qué ese aumento resulta engañoso en el escenario planteado.

d) Analiza los resultados mediante gráficos. Identifica el mejor y el peor modelo, comenta el efecto de los hiperparámetros y explica a qué atribuyes las diferencias entre validación y prueba.

NOTA: El no responder las preguntas de análisis implica un descuento, ya que hacen referencia al trabajo realizado en este item.

1.4.3. Combinación de ambos mundos (1.5 puntos)

Finalmente, deberás combinar la información visual y tabular para determinar si ambas fuentes se complementan. Recuerda relacionar los dos conjuntos mediante `filename` y conservar la partición definida en el Track A.

Combinación de características (0.5 puntos). Selecciona y combina las mejores características de las imágenes y de la ficha técnica. Justifica la elección considerando el desempeño previo, la redundancia y la información que aporta cada variable.

Entrenamiento y comparación (0.5 puntos). Vuelve a entrenar los mismos cuatro modelos con las mismas combinaciones de hiperparámetros utilizadas anteriormente. Realiza la selección con la estrategia de validación definida en el Track A y evalúa las configuraciones finales sobre el mismo conjunto de prueba.

Ranking final (0.5 puntos). Construye un listado con los **10 mejores modelos y sus hiperparámetros**, considerando los tres *tracks* y ordenándolos por desempeño de validación. Luego, evalúa los tres primeros lugares sobre el conjunto de prueba e incluye sus matrices de confusión.

Preguntas de análisis. Responde en `respuesta.pdf`:

a) ¿Por qué seleccionaste esas características visuales y tabulares para el modelo combinado?

b) ¿Obtuviste mejores o peores resultados que en los Tracks A y B? Identifica el mejor y el peor modelo, y explica las diferencias.

c) ¿Qué cambió respecto de los escenarios individuales? Discute si la información tabular aportó algo que la imagen no contenía o si resultó redundante o ruidosa.

d) ¿Qué observas en las matrices de confusión de los tres mejores modelos? Identifica qué piezas se confunden y explica por qué esos errores tienen sentido.

NOTA: El no responder las preguntas de análisis implica un descuento, ya que hacen referencia al trabajo realizado en este item.

¿Qué entregar en esta parte?

Para esta actividad se deberán entregar los siguientes archivos (dentro del directorio `01 - DCCubos`):

- `Tarea1_LEGO.ipynb` con todas sus celdas ejecutadas y el desarrollo completo. Mantén `piezas_dataset.csv` junto al notebook.
- `respuesta.pdf` con todas las respuestas de análisis, las figuras, las tablas de validación y prueba, el ranking final, las matrices de confusión y la declaración de uso de IA.
- No subir las imágenes descargadas ni la carpeta `data/` a canvas.

2. DCCUber: Aprendizaje Reforzado (6.0 puntos)

Has sido contratado por DCCUber, una aplicación de traslado en la que un usuario se contacta con un conductor para llegar a un destino seleccionado. Tu misión es salvar al peor conductor de DCCUber, cuya calificación se ha acercado peligrosamente a 1 estrella.

Tu tarea consiste en diseñar y entrenar un agente inteligente que sea capaz de recoger y dejar pasajeros en sus destinos alrededor de la ciudad, utilizando técnicas de *Reinforcement Learning*. El agente deberá aprender desde cero, únicamente a partir de la interacción con el entorno.

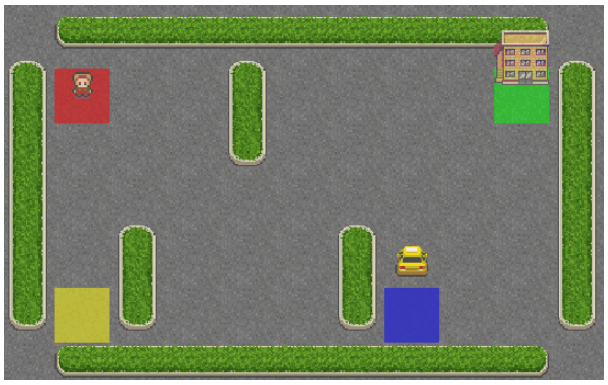


Figura 1: Simulación del entorno Taxi.

En tu repositorio se te entrega el código base del entorno, junto con un agente inicial que actúa de manera aleatoria. Deberás implementar el algoritmo *Q-Learning* para que el agente logre recoger y dejar pasajeros correctamente.

La interfaz utilizada para entrenamiento no es la misma que la de visualización. Durante el entrenamiento, el agente interactúa directamente con el entorno sin renderizado para acelerar el proceso.

Los archivos contenidos en el código base (dentro del directorio `02 - DCCUber`) son:

- `main.py`: Archivo principal de ejecución. **No modificar.**
- `train.py`: Contiene el loop de entrenamiento. **Modificar**
- `test.py`: Permite evaluar el agente entrenado. **No modificar.**
- `TaxiAgent.py`: Contiene la clase del agente que se mueve de manera aleatoria. Es aquí donde deberás trabajar. **Modificar**

Como ya se mencionó, el archivo sobre el cual debes trabajar es `TaxiAgent.py`[1]. Este archivo contiene la clase `TaxiAgent` con los siguientes métodos:

- `__init__()`: Inicializa los atributos del agente. En el caso del agente aleatorio, solo se inicializan en 0 las partidas jugadas por el agente.
- `get_state(observation)`: Consulta por el estado actual del agente y lo retorna como un valor entre 0 a 499. Cada uno de estos valores representa una combinación distinta de:

    1. **Posición actual del vehículo** (x_t, y_t): Corresponden a la posición del vehículo en la grilla. Tanto x_t como y_t toman valores entre 0 a 4 dentro de la grilla.

[1] Este archivo se encuentra comentado, por lo que se recomienda leerlo como parte del enunciado

2. **Ubicación actual del pasajero (p)**: Toma un valor entero entre 0 y 4 que corresponde a las 5 posibles ubicaciones de pasajeros.
3. **Destino al que debe ser transportado (d)**: Toma un valor entero entre 0 y 3 que corresponde a los 4 posibles destinos del viaje.

Nota: Como se mencionó anteriormente, el valor final entregado corresponde a una posible combinación de los valores mencionados (25 posibles posiciones del vehículo $*$ 5 posibles posiciones de pasajeros $*$ 4 posibles destinos $= 500$), por lo que la Q-Table esperada usa únicamente este valor como índice, en lugar de utilizar x_t, y_t, p, d como índices separados.

- `get_action(state)`: Recibe el estado del agente y retorna un entero representando la acción que debe seleccionar el agente. Las acciones posibles son:
  - `0`: Moverse hacia abajo.
  - `1`: Moverse hacia arriba.
  - `2`: Moverse hacia la derecha.
  - `3`: Moverse hacia la izquierda.
  - `4`: Recoger al pasajero.
  - `5`: Dejar al pasajero.

  Nota: En el caso del agente aleatorio, este no utiliza la información que entrega el estado y retorna una acción al azar.
- `update_q_value(state, action, reward, next_state, done)`: Actualiza la *Q-Table* para una tupla (estado, acción) usando *Q-Learning*.
- `decay_exploration_rate()`: Actualiza la tasa de exploración una vez por episodio.

IMPORTANTE

Para participar en el campeonato, tu código debe generar la *Q-Table* de tu agente ya entrenado como un archivo llamado `q_table_campeonato.csv`, sin *headers* (esto es, filas con títulos).

El archivo `.csv` debe poseer 7 columnas y 500 filas. La primera columna corresponde a los valores de las dimensiones de los estados en el orden descrito en este enunciado. Por otra parte, las siguientes 6 columnas corresponden al *Q-Value* de las acciones posibles del agente en el orden del enunciado. De esta forma, cada fila representa un posible estado y los valores de las acciones posibles para este estado. Una fila de este archivo se debe ver así:

$$[s,\ Q_0,\ \ Q_1,\ \ Q_2,\ \ Q_3,\ \ Q_4,\ Q_5]$$

Donde s representa una combinación de los parámetros ($x_t,\ \ y_t,\ \ p,\ \ d$).

Actividad 1: Comprendiendo los hiperparámetros (1 punto)

Nota: Para esta actividad no es necesario tocar el código.

1. Investiga la ecuación de actualización de Q-Learning:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max Q(s',a') - Q(s,a)]$$

   Explica la importancia de los hiperparámetros α y γ dentro de la ecuación de actualización de Q-Learning.
   Analiza por separado el efecto de cada uno:

a) ¿Qué diferencias esperarías en el aprendizaje al utilizar $\alpha = 0.1$ y $\alpha = 1$, manteniendo γ constante? Considera tanto la velocidad como la estabilidad del aprendizaje. **(0.1 puntos)**

b) ¿Qué diferencias esperarías al utilizar $\gamma = 0.1$ y $\gamma = 1$, manteniendo α constante? Explica cómo cambia la importancia que el agente asigna a las recompensas futuras. **(0.1 puntos)**

2. Dado que se tienen los agentes A y B, con valores $\alpha_A = 0.9, \gamma_A = 0.1, \alpha_B = 0.2, \gamma_B = 0.8$, responda las siguientes preguntas:

   a) ¿De cuál de los agentes se esperaría un aprendizaje más rápido? ¿Cuál presentaría un aprendizaje más gradual y estable a lo largo de los episodios? Justifique a partir de los valores de α_A y α_B. **(0.1 puntos)**

   b) ¿Cuál de los agentes considera más las consecuencias de acciones futuras al momento de tomar una decisión? ¿Cómo tomar en consideración el futuro puede ayudar a mejorar el rendimiento de un agente? Justifique a partir de los valores de γ_A y γ_B. **(0.1 puntos)**

   c) Tomando en consideración lo respondido anteriormente: ¿De qué agente se esperarían mejores resultados despues de realizar un entrenamiento con 10000 episodios? Justifique su respuesta considerando la velocidad y estabilidad del aprendizaje, además de la importancia otorgada a las recompensas futuras. **(0.1 puntos)**

   d) En el contexto de *TaxiAgent*: ¿Qué implicaría utilizar cada uno de los agentes A y B al momento de aprender a desplazarse por la ciudad y evitar movimientos poco eficientes? Explique cómo los valores de α y γ podrían influir en la rapidez con que aprenden rutas y en la capacidad de considerar las consecuencias futuras de sus movimientos. **(0.1 puntos)**

3. Investiga cómo funciona la exploración ϵ-greedy y responde las siguientes preguntas:

   a) Explica como funcionan las etapas de exploración y explotación en ϵ-greedy. ¿Qué determina la probabilidad de que el agente explore o explote en cada decisión? **(0.1 puntos)**

   b) Considere a partir de ahora la siguiente fórmula para el decaimiento de ϵ:

   $$\epsilon = \epsilon_{inicial} * decay^{n_de_episodios}$$

   ¿Qué efecto tiene usar un valor de *decay* cercano a 1 sobre la velocidad con que disminuye ϵ? **(0.1 puntos)**

   c) Si $\epsilon_{inicial} = 1$ y $decay = 0.995$ calcule el valor de ϵ después de 1000 episodios. **(0.1 puntos)**

   d) ¿Qué relación existe entre la tasa de aprendizaje α y la tasa de exploración ϵ durante el entrenamiento de un agente? **(0.1 puntos)**

Actividad 2: Implementando Q-Learning (1.5 puntos)

Básandote en lo visto en clases, implementa el algoritmo *Q-Learning*[2] para el agente del taxi. Para esto, deberás realizar los siguientes pasos en `TaxiAgent.py`[3]:

1. Inicializar la *Q-Table* en el método `__init__()` con sus dimensiones correctas **(0.4 puntos)**.

2. Modificar el método `get_action()` para que el agente sea capaz de explorar el estado actual o explotar la mejor acción conocida hasta el momento **(0.6 puntos)**.

3. Modificar el método `update_q_value()` para actualizar la *Q-Table* una vez que se ejecute la acción seleccionada por el agente **(0.5 puntos)**. La función debe:

[2]Si tienes dudas con *Q-Learning*, puedes revisar este artículo.
[3]Se recomienda revisar los `#TODO` que hay en el código.

a) Buscar los índices de los estados en la tabla de estados. **No modificar**.

b) Calcular el mejor valor Q del siguiente estado (o 0 si el episodio terminó).

c) Calcular el objetivo temporal (`td_target`) sumando la recompensa y el descuento del mejor siguiente Q.

d) Calcular el error temporal (`td_error`) restando el valor Q actual del objetivo.

e) Ajusta el valor Q actual en la tabla usando la tasa de aprendizaje `LR`.

f) Guardar el error de entrenamiento para seguimiento. **No modificar**.

Actividad 3: Experimentación y Análisis del Agente (2.5 puntos)

Para esta sección, deberás realizar cuatro experimentos distintos modificando los hiperparámetros del agente. Para cada experimento, debes **generar evidencia** en el archivo `train.py` y usarla para fundamentar tus respuestas. Cuando se pidan tablas o gráficos se pedirá que se adjunten en un documento `respuesta.pdf`.
Nota: Utilice al menos 5000 episodios a menos de que se indique lo contrario.

- **Experimento 1: El Agente "Miope"vs. el Agente "Previsor"(0.5 puntos)**

  En este experimento se probaran los efectos de un agente que prioriza recompensas inmediatas versus uno que considera las decisiones futuras. Para ello, deberás seguir los siguientes pasos:

  1. Realiza dos entrenamientos: uno con el valor original ($\gamma = 0$) y otro con un valor alto ($\gamma = 1$) que incentive al agente a ser "previsor".
  2. Genera un gráfico que muestre el `mean_score` (puntaje promedio) para **cada entrenamiento** en la misma figura.
  3. Analice los gráficos, responde: ¿Qué agente aprende más rápido al principio? ¿Cuál alcanza un mejor rendimiento máximo a largo plazo? ¿Cómo la evidencia visual de tu gráfico demuestra la diferencia fundamental entre un agente que busca recompensas inmediatas y uno que planifica para el futuro?
  4. Pruebe al menos 3 valores entre 0 y 1 distintos de γ y genere un gráfico que muestre el puntaje promedio para **cada entrenamiento** en la misma figura.
  5. Finalmente, en base a lo discutido anteriormente, ¿qué valor de γ elegirías?

- **Experimento 2: El Dilema de la Velocidad vs. la Estabilidad (0.5 puntos)**

  El `LR` (Learning Rate) no solo afecta qué tan rápido aprende el agente, sino que también qué tan estable es su conocimiento. Para este experimento, deberás seguir los siguientes pasos:

  1. Realiza un experimento comparando un `LR` muy bajo (ej. `0.01`) contra un `LR` muy alto (ej. `0.7`).
  2. Para cada caso, registra el `record` (puntaje máximo) y el `mean_score` cada 100 episodios. Presenta tus resultados en una tabla comparativa.
  3. Basado en la tabla, compara el proceso de aprendizaje del agente con tasas de aprendizaje (`LR`) bajas y altas. Describe la velocidad y las características de su rendimiento (p. ej., estable, errático, estancamiento) en cada escenario. Finalmente, selecciona el valor de `LR` que consideres óptimo y justifica tu elección basándote en el compromiso (*trade-off*) observado entre la velocidad de convergencia y la estabilidad del rendimiento.

- **Experimento 3: Saboteando el Aprendizaje (0.5 puntos)**

  El balance entre explorar el entorno y explotar el conocimiento adquirido es fundamental. En este experimento, investigaremos qué pasa cuando ese balance se rompe. Para ello, deberás seguir los siguientes pasos:

  1. Diseña y ejecuta dos experimentos "fallidos":
     a) **Exploración Prematura**: Modifica el `EXPLORATION_DECAY_RATE` a un valor muy alto (ej. `0.05`) para que el agente deje de explorar casi inmediatamente.
     b) **Exploración Eterna**: Modifica el `MIN_EXPLORATION_RATE` para que sea igual a `MAX_EXPLORATION_RATE` (ej. `0.8`), forzando al agente a tomar una acción aleatoria el 80 % de las veces, para siempre.
  2. Describe el comportamiento observado en los escenarios de "Exploración Prematuraz "Exploración Eterna". Evalúa la calidad de las políticas aprendidas y fundamenta cómo el balance entre exploración y explotación en cada caso explica el rendimiento final del agente.
  3. Explica cómo los resultados de estos dos experimentos demuestran por qué es crucial tener una tasa de exploración que **decaiga gradualmente** con el tiempo.

- **Experimento 4: La Curva de la Experiencia (0.5 puntos)**

  La cantidad de episodios de entrenamiento determina cuánta experiencia puede acumular el agente antes de ser evaluado. Sin embargo, entrenar durante más tiempo no necesariamente implica una mejora proporcional en su rendimiento. Para este experimento, deberás seguir los siguientes pasos:

  1. Entrena agentes utilizando distintas cantidades de episodios: 500, 2500, 5000, 10000 y 20000.
  2. Para cada cantidad de episodios, registra el *mean score* final y el puntaje máximo alcanzado. Presenta los resultados en una tabla comparativa.
  3. Compara el rendimiento obtenido al aumentar la cantidad de episodios de entrenamiento. Identifica a partir de qué cantidad de episodios las mejoras comienzan a ser menores o el rendimiento tiende a estabilizarse.
  4. Basado en los resultados, explica si entrenar durante más episodios siempre produce un agente significativamente mejor.

- **Experimento 5: Entrenando a tu Agente Ideal (0.5 puntos)**

  Hemos descubierto que los hiperparámetros del agente dictan su desempeño (ya sea para bien o para mal) y que más episodios no siempre significan mejoras muy significativas. Para este último experimento, diseñaremos y entrenaremos a nuestro "agente ideal". Para ello, deberás seguir los siguientes pasos:

  1. Selecciona y justifica brevemente la configuración de hiperparámetros que consideres óptima y reentrena al agente con ella.
  2. **Genera un gráfico** que muestre el `mean_score` (puntaje promedio) a lo largo del tiempo del entrenamiento.
  3. Reflexiona sobre el rendimiento de tu agente. Considera si un agente "perfecto" es un objetivo realista en todos los casos y describe qué acciones específicas tomarías para mejorar aún más los resultados obtenidos, justificando tu propuesta.

Actividad 4: SlimeVolleyGym (1.0 puntos)

`slimevolleygym` (repositorio de David Ha / hardmaru) es un entorno de Gym liviano que simula un juego de vóleibol 2D entre "slimes" (dos jugadores con forma de gota). El objetivo de cada agente es hacer que la pelota toque el suelo del lado contrario, evitando que toque el suelo propio.

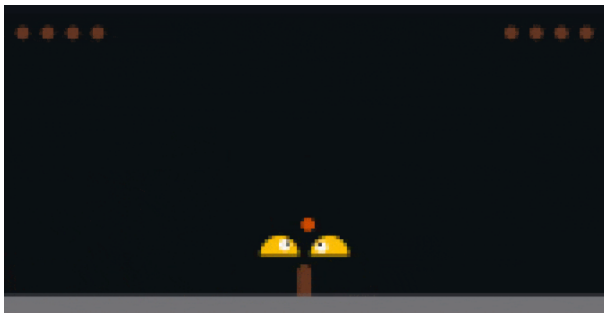


Figura 2: Simulación del entorno de Slime.

Cómo funciona a grandes rasgos:

- **Física simple 2D**: hay un agente a cada lado de una red, una pelota que rebota, y gravedad. El objetivo es anotar puntos hasta que el rival se quede sin vidas (por defecto, 5 vidas) o se llegue al máximo de pasos del episodio (por defecto, 3000).
- **Espacios de observación**: el repositorio ofrece dos modalidades:
  - `SlimeVolley-v0`: observación en forma de **vector continuo de 12 dimensiones**, con las posiciones (x, y) del propio agente, del rival y de la pelota, en dos frames consecutivos (posición actual y "siguiente"/velocidad implícita). Es decir, el estado es un vector de números reales, no un índice discreto.
  - `SlimeVolleyPixel-v0` / `SlimeVolleyNoFrameskip-v0`: observación como **imagen RGB** ($84 \times 168 \times 3$), al estilo de los entornos de Atari.
- **Espacio de acciones**: `MultiBinary(3)`, es decir, tres botones que pueden activarse en combinación (adelante, atrás, saltar), lo que da 6 combinaciones posibles de movimiento. No es un único entero discreto como en Taxi.
- **Recompensa**: $+1$ si el rival deja caer la pelota, -1 si la deja caer el propio agente, 0 mientras el punto sigue en juego. Es una recompensa bastante *dispersa* (sparse) comparada con la de taxi.
- **Pensado para self-play**: el mismo entorno permite fácilmente que un agente juegue contra sí mismo (versiones pasadas de su propia política), lo que lo hace un benchmark clásico para algoritmos de *self-play* y multiagente, además de single-agent contra un bot base ya entrenado que viene incluido en el repo.

Responda las siguientes preguntas. Puedes revisar el repositorio oficial de SlimeVolleyGym para más información sobre el entorno[4].

1. Compara explícitamente los espacios de estado de **Taxi-v4** y **SlimeVolley-v0**: Taxi-v4 tiene 500 estados discretos, enumerables e indexables directamente (0 a 499); SlimeVolley-v0 tiene un vector continuo de 12 dimensiones (`float64`) con las posiciones de agente, rival y pelota. ¿Qué implica esta diferencia para la forma en que se puede representar una política? **(0.2 puntos)**

[4] `https://github.com/hardmaru/slimevolleygym`

2. ¿Por qué no sería razonable aplicar el mismo enfoque de Q-Learning tabular (una Q-Table indexada por estado) que usaste en Taxi-v4 directamente al vector de 12 dimensiones de SlimeVolley-v0? Argumenta considerando el tamaño y la naturaleza (continua vs. discreta) del espacio de estados. **(0.2 puntos)**

3. ¿Por qué SlimeVolleyGym usa redes neuronales? Los agentes de referencia de este repositorio están implementados con redes neuronales en lugar de una Q-Table. Considerando lo discutido en la pregunta anterior sobre el tamaño y la naturaleza del espacio de estados, ¿por qué una red neuronal es una elección natural para este entorno? **(0.2 puntos)**

4. ¿Qué reemplaza exactamente la red neuronal respecto a la Q-Table que implementaste para Taxi-v4? Explica qué recibe como entrada, qué entrega como salida, y por qué esto le permite prescindir de una tabla que enumere cada estado posible. **(0.2 puntos)**

5. Menciona **una ventaja concreta** de usar una red neuronal en lugar de una Q-Table para representar la política de un agente similar a SlimeVolley-v0. **(0.2 puntos)**

Bonus: DCCampeonato IIC2613 2026-2

Celebraremos la **cuarta edición** del DCCampeonato IIC2613, en donde pueden poner a prueba sus agentes de aprendizaje reforzado y competir con sus demás compañeros de curso en un torneo por la mejor calificación en DCCUber. Los primeros 5 lugares **podrán ganar hasta 10 décimas** en su tarea, además de entrar al salón de la fama del curso.



El requisito para participar de este campeonato es:

1. Completar la función `DCCampeonato()` en `train.py`, la cual deberá entrenar un agente utilizando los hiperparámetros que estimen convenientes, empleando como máximo 650 episodios. Al ejecutar esta función desde `main.py`, esta debe generar automáticamente el archivo `q_table_campeonato.csv` correspondiente a la Q-Table resultante del entrenamiento, guardado en el mismo directorio de ejecución.

¿Qué entregar en esta parte?

Se deberá entregar los archivos `train.py` y `TaxiAgent.py` modificados y un archivo `respuesta.pdf` con tus respuestas, todo dentro del directorio `02 - DCCUber`.

Referencias

D. Ha, "Slime Volleyball Gym Environment," GitHub repository, 2020. `https://github.com/hardmaru/slimevolleygym`